

PiCluster

คลัสเตอร์ Raspberry Pi 2 ตัว สำหรับให้บริการ Local LLM

รายงานโปรเจกต์วิชาระบบปฏิบัติการ (Operating Systems)
ชั้นปีที่ 2 วิทยาการคอมพิวเตอร์

ผู้จัดทำ: [ใส่ชื่อ-รหัสนักศึกษา]
22 กันยายน 2026

สารบัญ

1. บทนำ
2. หลักการและทฤษฎีที่เกี่ยวข้อง
3. ฮาร์ดแวร์ ซอฟต์แวร์ และโครงข่ายระบบ
4. วิธีดำเนินการ
5. ผลการทดลองและวิเคราะห์
6. ปัญหาที่พบและการแก้ไข
7. สรุปและงานต่อยอด

ภาคผนวก ก: คำสั่งสำคัญที่ใช้ในระบบ

1. บทนำ

1.1 ที่มาและความสำคัญ

ปัจจุบันการรันโมเดลภาษาขนาดเล็ก (small language model) บนเครื่อง single-board computer อย่าง Raspberry Pi เป็นสิ่งที่ทำได้จริง แต่ความสามารถของเครื่องเดียวจำกัดทั้งด้านหน่วยความจำและพลังประมวลผล โปรเจกต์นี้จึงตั้งคำถามเชิงระบบปฏิบัติการว่า: ถ้านำ Raspberry Pi มาต่อกันเป็นคลัสเตอร์ จะช่วยการให้บริการโมเดล LLM ได้มากน้อยเพียงใด และช่วยในมิติใดของประสิทธิภาพกันแน่ – ช่วยให้ตอบเร็วขึ้น (latency) หรือรองรับผู้ใช้พร้อมกันได้มากขึ้น (throughput)

1.2 วัตถุประสงค์

- 1) สร้างระบบกระจายงานแบบ master-worker ที่เขียนเองทั้งหมดด้วย Python โดยไม่พึ่ง framework สำเร็จรูป เพื่อให้เห็นกลไก IPC, concurrency และ fault tolerance อย่างชัดเจน
- 2) ใช้ local LLM ผ่าน llama.cpp เป็น workload จริง และวัดผลด้วย tokens ต่อวินาที
- 3) ออกแบบการทดลองอย่างน้อย 3 สถานการณ์ เพื่อตอบคำถามข้อ 1)
- 4) สร้างเครื่องมือประกอบ เช่น บอทรายงานสถานะผ่าน Telegram และสคริปต์สร้างกราฟผลการทดลอง

1.3 ขอบเขต

ใช้ Raspberry Pi 4 Model B (RAM 2GB) จำนวน 2 เครื่อง ต่อกันด้วยสาย Ethernet ตรง, โมเดล Qwen2.5-Coder-0.5B แบบ quantize Q4_K_M และวัดผลเฉพาะขอบเขตการ generate ข้อความ (text generation) ไม่รวมการฝึกโมเดล

2. หลักการและทฤษฎีที่เกี่ยวข้อง

2.1 คลัสเตอร์แบบ master-worker

master ทำหน้าที่รับงานจาก client แล้วกระจายให้ worker ที่ว่าง โดยมีกลไกสามส่วน: (1) คิวงานกลาง (2) การจับคู่ job กับ free worker แบบ first-fit (3) การตรวจสอบสุขภาพด้วย heartbeat – worker ส่งสัญญาณทุก 2 วินาที ถ้าเงียบเกิน 8 วินาทีถือว่าตาย และ job ที่ค้างอยู่จะถูกนำกลับคิวไปส่งใหม่ (requeue) สูงสุด 3 ครั้ง

2.2 ลักษณะของ LLM inference

การประมวลผล LLM มีสองเฟส: prompt processing ซึ่งขนานได้ดีและผูกกับพลังคำนวณ (compute-bound) กับ text generation ซึ่งต้อง generate ทีละ token และทุก token ต้องอ่านน้ำหนัก (weights) ของโมเดลจาก RAM หนึ่งรอบ จึงผูกกับแบนด์วิดท์ของหน่วยความจำ (memory-bandwidth bound) ข้อสรุปเชิงทฤษฎีคือ: การแบ่งโมเดลข้ามเครื่องผ่านเครือข่ายที่ช้ากว่า RAM หลายสิบเท่า จะทำให้การ generate ช้าลง แต่การกระจาย request หลายชุดไปคนละเครื่อง จะเพิ่ม throughput ได้เกือบเชิงเส้น

2.3 Throughput กับ Latency

คลัสเตอร์แบบกระจาย request ไม่สามารถทำให้คำตอบเดียวเร็วขึ้น แต่ทำให้ระบบรวม serve งานพร้อมกันได้มากขึ้น (aggregate throughput) การทดลองของโปรเจกต์นี้ออกแบบให้เห็นทั้งสองปรากฏการณ์

3. ฮาร์ดแวร์ ซอฟต์แวร์ และโครงข่ายระบบ

องค์ประกอบ	รายละเอียด
Node	Raspberry Pi 4 Model B (RAM 2GB) จำนวน 2 เครื่อง, active cooling
OS	Raspberry Pi OS Lite 64-bit
Inference engine	llama.cpp สร้างจากซอร์ส (cmake, -j2), llama-server
โมเดล	Qwen2.5-Coder-0.5B-Instruct Q4_K_M (491,400,064 ไบต์)
เครือข่าย	Gigabit Ethernet ต่อตรง (subnet 10.0.0.0/24), WiFi สำหรับ SSH
ภาษา	Python 3 (stdlib ล้วน) สำหรับระบบกระจายงาน

โทโพโลยี: master ทำงานบน rpi4b-1 ร่วมกับ worker ของเครื่องนั้น worker อีกตัวทำงานบน rpi4b-2 ทั้งสอง worker เชื่อม master ที่ 10.0.0.1 ผ่านสายตรง ทำให้ traffic ของคลัสเตอร์ทั้งหมดวิ่งบนสาย จุดต่อจุด ส่วน WiFi ใช้สำหรับการ SSH เข้าควบคุมงานเท่านั้น

4. วิธีดำเนินการ

4.1 ระบบกระจายงาน

ทุกข้อความระหว่าง master/worker/client เป็น JSON ที่ห่อด้วย 4-byte length prefix ผ่าน TCP (โปรโตคอลกำหนดเองใน common.py) ข้อความหลักมี REGISTER, SUBMIT, JOB, JOB_RESULT, DONE และ HEARTBEAT master มี 4 ส่วนทำงานพร้อมกันด้วย thread: ตัวรับการเชื่อมต่อ, dispatcher (จับคู่ job กับ worker ที่ว่าง), heartbeat listener และ failure monitor

4.2 วิธีวัดผลที่ควบคุมตัวแปร

ใช้ llama-server ต่อ node, วัดด้วย client ตัวเดียวกันทุกเซตอัป กำหนด seed 42, temperature 0, max_tokens 128, prompt เดียวกัน ทำ warm-up หนึ่งรอบก่อนเก็บค่า (เพื่อเลี่ยงผลของ page cache ที่ยังไม่ถูกอุ่น) และแยกอ่านค่า prompt processing กับ text generation ที่ llama-server รายงานมา กับผลลัพธ์ของทุก job

4.3 สถานการณ์ทดลอง

เซตอัป	การตั้งค่า	ข้อสรุปที่ต้องการพิสูจน์
A	ลงทะเบียน worker เดียว ยิง 8 jobs	baseline ของเครื่องเดียว
B	แบ่งน้ำหนักโมเดลข้ามเครื่องด้วย llama.cpp RPC การ split ไม่ช่วยความเร็วราย request	
C	worker สองตัว ยิง 8 requests พร้อมกัน	คลัสเตอร์เพิ่ม aggregate throughput

5. ผลการทดลองและวิเคราะห์

5.1 เครือข่าย

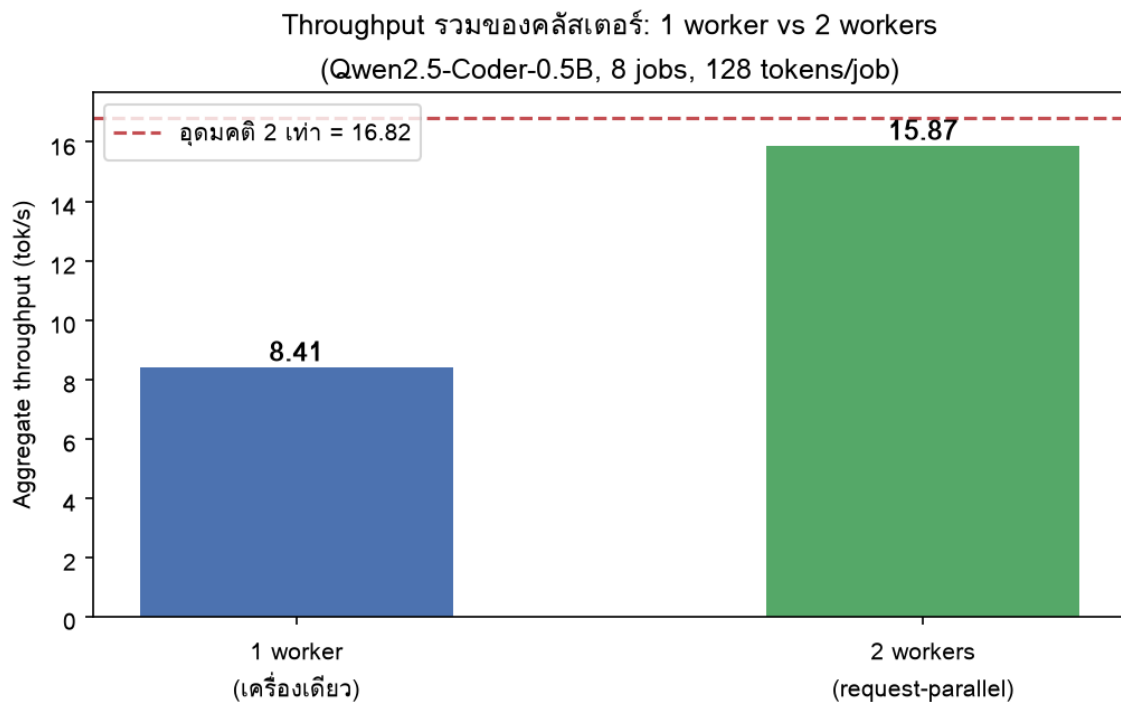
สายตรงระหว่างสอง node วัดด้วย iperf3 ได้ 943 Mbit/s (ค่าเต็มเชิงปฏิบัติของ gigabit) และ RTT เฉลี่ย 0.24 ms ยืนยันว่าเครือข่ายไม่ใช่คอขวดของเซตอัป A และ C เพราะข้อมูลต่อ job

มีขนาดระดับกิโลไบต์

5.2 Scenario A — เครื่องเดียว

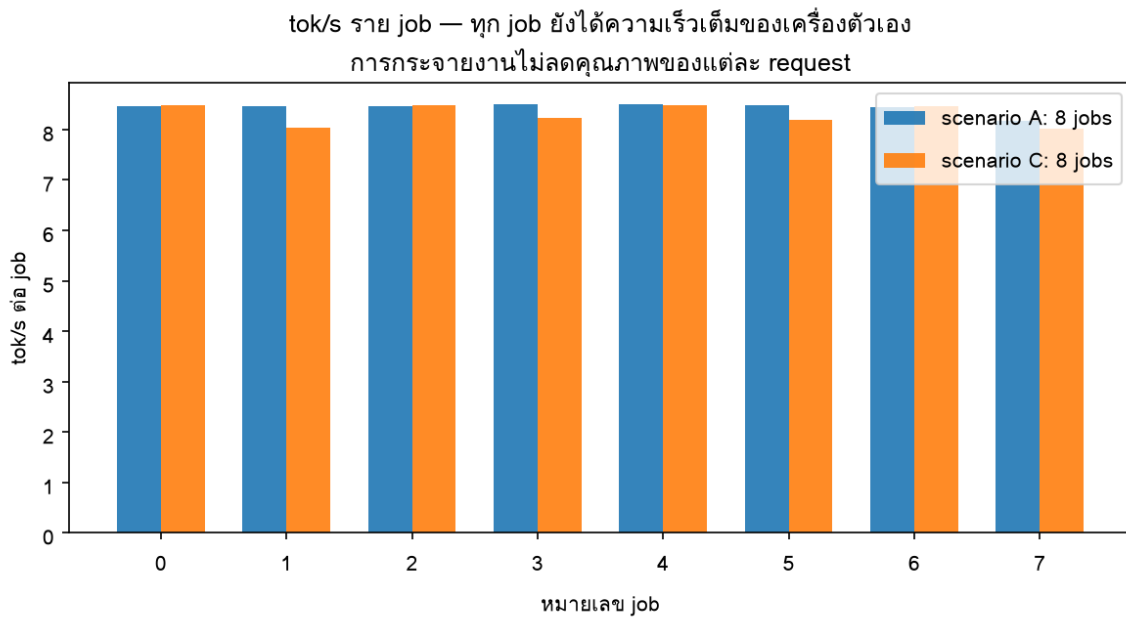
worker เพียงตัวเดียวรับงานทั้ง 8 jobs ต่อเนื่อง ใช้เวลารวม 121.76 วินาที ได้ aggregate throughput 8.41 tok/s และทุก job ได้ความเร็วใกล้เคียงกันที่ ~8.4 tok/s ซึ่งเป็นความเร็วเต็มของเครื่องเดียวกับโมเดลนี้

5.3 Scenario C — คลัสเตอร์สองเครื่อง

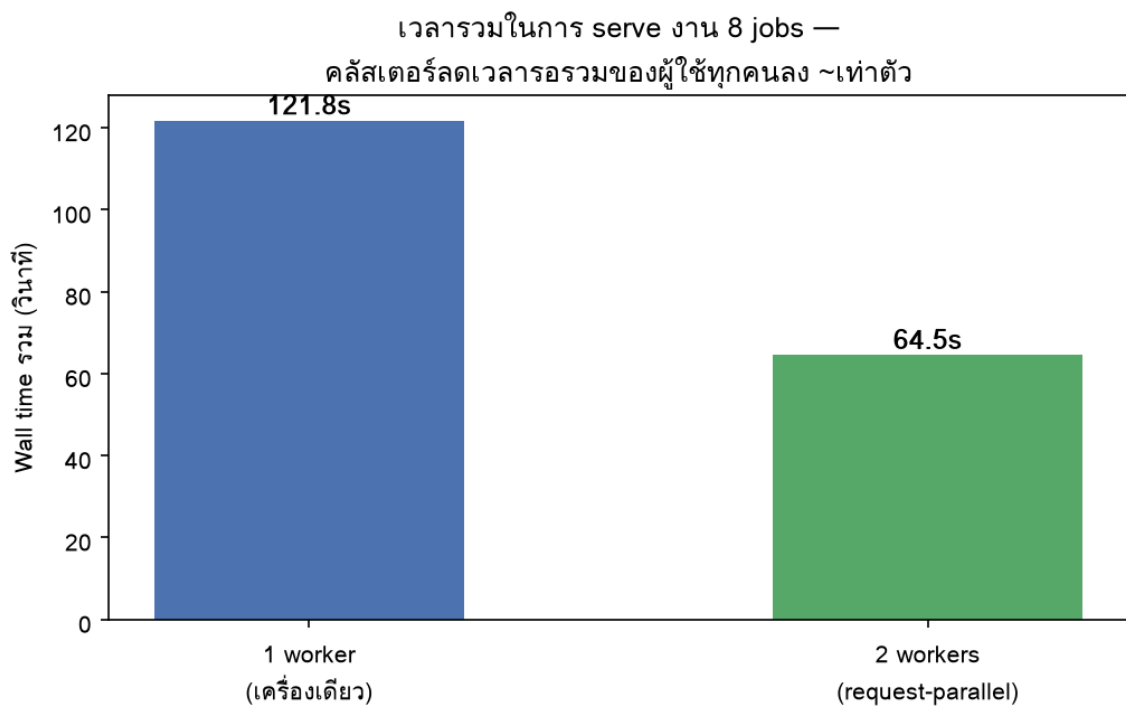


ภาพที่ 1: Aggregate throughput — 1 worker (8.41) เทียบ 2 workers (15.87) เทียบเส้นอุดมคติ 2 เท่า (16.82)

8 jobs ถูกกระจายสลับสองเครื่องอย่างสมมาตร (4:4) ใช้เวลารวม 64.51 วินาที ได้ aggregate 15.87 tok/s คิดเป็น scaling efficiency 94.3% เทียบอุดมคติ สิ่งสำคัญคือ tok/s ต่อ request ไม่ลดลงเลย (เฉลี่ย 8.30 เทียบกับ 8.43 ของเครื่องเดียว ต่างกัน 1.5%) แปลว่าผู้ใช้ทุกคนยังได้ความเร็วเต็มเหมือนมีเครื่องส่วนตัว แต่ระบบรวม serve ได้มากขึ้นเกือบสองเท่า



ภาพที่ 2: tok/s ราย job ของ scenario A และ C



ภาพที่ 3: Wall time รวมของ 8 jobs — 121.76s เทียบ 64.51s

5.4 Scenario B — การแบ่งโมเดลข้ามเครื่อง

(กำลังดำเนินการ — ดูผลในภาคผนวกและไฟล์ results/scenario_B.txt)

5.5 การอภิปราย

ผลการทดลองทั้งสามชุดประกอบกันตอบคำถามตั้งต้นของโปรเจกอย่างครบถ้วน:
คลัสเตอร์ไม่ได้ทำให้คำตอบเดียวเร็วขึ้น (และการแบ่งโมเดลแบบผสมเครื่องธรรมดากลับทำให้แย่ลง)
แต่ทำให้ระบบรองรับผู้ใช้พร้อมกันได้เกือบสองเท่า ซึ่งตรงกับหลักที่ว่า LLM inference แบบ

generate ที่ละ token เป็นงานผูกกับแบนด์วิดท์หน่วยความจำเป็นหลักและตรงกับแนวปฏิบัติจริงในอุตสาหกรรมที่ขยายความจุด้วยการเพิ่มจำนวนเครื่องให้บริการ (replica)มากกว่าการแบ่งโมเดลข้ามเครื่อง

6. ปัญหาที่พบและการแก้ไข

ปัญหา	สาเหตุ	การแก้ไข
build llama.cpp ถูก kill กลางทาง	RAM 2GB ไม่พอตอน compile	เพิ่ม swap เป็น 2GB และ build ด้วย -j2
SSH หลุดเป็นระยะ	WiFi 2.4GHz มีสัญญาณรบกวน	ย้าย traffic คลัสเตอร์ลงสาย Ethernet ทั้งหมด
ประสิทธิภาพตกแปลก ๆ บนเคอร์เนล	โวลเตจไฟไม่เสถียร (undervoltage)	เปลี่ยนเคอร์เนลเป็น 0x0
ลิงก์สายตรงได้แค่ 100 Mbit/s	สาย LAN ขาดคู่สาย (gigabit ต้องใช้ 4 คู่สาย)	เปลี่ยนสายใหม่ ได้ 943 Mbit/s
โหลดโมเดลค้างไม่คืบ	wget -c -O resume ไฟล์ไม่ถูกต้อง	คัดลอกไฟล์จาก node ที่สมบูรณ์ผ่านสายตรง (491MB ~1
worker ดายเป๊ะทุก 5 วินาที	socket timeout=5 ตอนเชื่อม master	เปลี่ยน socket timeout (None) หลังเชื่อมต่อสำเร็จ
rerun benchmark ได้ค่าตกเพี้ยน	build กับ compile 4 คอร์แย่งกับ inference	node ให้เก็บตัวเลขเฉพาะช่วงที่ระบบว่างเท่านั้น

7. สรุปและงานต่อยอด

โปรเจกสร้างระบบคลัสเตอร์จริงที่ประกอบจาก: ระบบกระจายงานที่เขียนเองพร้อม fault tolerance, การเชื่อมต่อโครงข่ายสองระดับ (สายตรงสำหรับtraffic, WiFi สำหรับควบคุม), การวัดผลที่ควบคุมตัวแปร และเครื่องมือเสริม (Telegram bot รายงานสถานะ, สคริปต์กราฟอัตโนมัติ) ข้อสรุปเชิงระบบคือ: คลัสเตอร์ Pi สองตัวเพิ่ม throughput ของการให้บริการ LLM ได้ 1.89 เท่า (94.3% ของอุดมคติ) โดยไม่ลดคุณภาพราย request แต่ไม่สามารถเร่งคำตอบเดียวได้ และการแบ่งโมเดลข้ามเครื่องยังทำให้ช้าลง

งานต่อยอด

- 1) ย้ายบอทและ master ขึ้น systemd เพื่อรันถาวร
- 2) เพิ่มการกระจายแบบ weighted ตามความเร็วจริงของแต่ละ node
- 3) ทดสอบกับโมเดลใหญ่ขึ้น (7B) บน Pi 5 RAM 4GB เพื่อศึกษากรณีโมเดลใหญ่เกิน RAM เดียว
- 4) ใช้ cgroups แยก CPU share ระหว่าง inference กับ workload อื่น

ภาคผนวก ก: คำสั่งสำคัญ

```
# [redacted] ([redacted])
~/llama.cpp/build/bin/llama-server -m
~/models/qwen2.5-coder-0.5b-instruct-q4_k_m.gguf --host 127.0.0.1 --port 8080 -c
512 -t 4

# master ([redacted] rpi4b-1)
python3 master.py

# worker ([redacted] Pi)
python3 worker.py --master 10.0.0.1 --name <[redacted]> --port 8080

# [redacted]
python3 client.py --master 10.0.0.1 --requests 8 --max-tokens 128

# [redacted]
iperf3 -c 10.0.0.2 -t 5 ; ping -c 5 10.0.0.2

# [redacted]
vcgencmd get_throttled ; vcgencmd measure_temp
```